



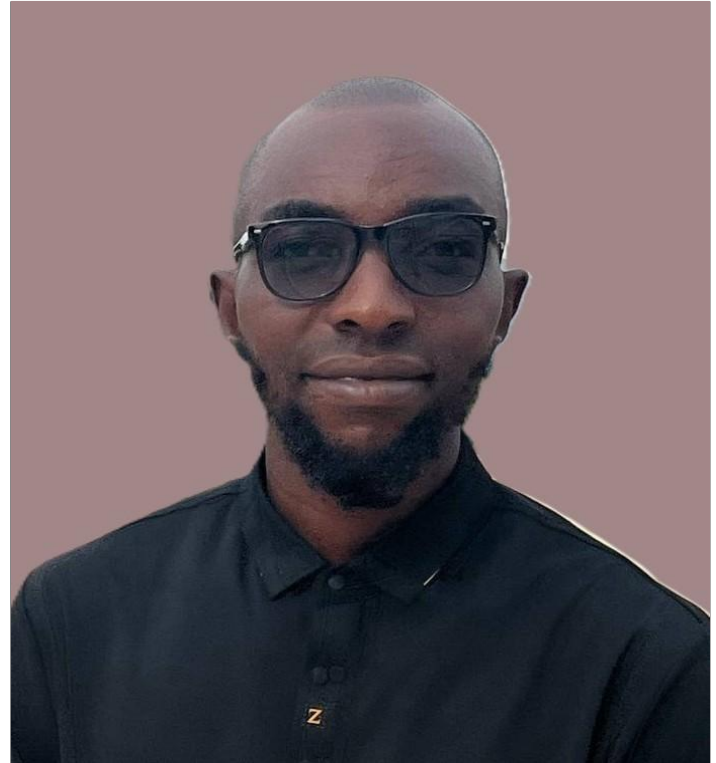
Lisk Summer Bootcamp

Etette Etok

Software Engineer & Web3 DevRel

Twitter: @Em_Etok

Linkedin: Etette Etok



Today's Agenda

- Foundry
- Inheritance
- Libraries
- Events
- Errors and Error handling



Foundry

Prerequisites

- Node.js and npm
- Git

Installation Steps

- Install Foundry using the command: `curl -L https://foundry.paradigm.xyz | bash`
- Set up environment variables: `source ~/.bashrc` or `source ~/.zshrc`

Configuration

- Configuring Foundry: run `<foundryup>` command
- Initializing a new project: run `forge init "my_project_name"`

Directory Structure

- `src`: Contains source code
- `test`: Contains test files
- `lib`: For dependencies
- `Script`: contain script files
- `out`: Compiled contract artifacts

Foundry

Foundry is a powerful platform designed for developers to build and deploy decentralized applications on the Ethereum blockchain. It is a comprehensive toolbox that provides all the necessary tools and equipment to build, test and deploy your decentralized applications efficiently. It includes three main tools: Forge, Cast and Anvil.

Forge:

- Initialize a new foundry project
- Compiles Solidity contracts.
- Runs unit and fuzz tests to ensure contract reliability.
- Deploys contracts to networks (testnets or mainnet).

Cast:

- Sends transactions or calls functions on smart contracts.
- Queries blockchain data (contract state, balances etc).
- Converts data formats (e.g hex to decimal).

Anvil:

- Simulates a full Ethereum blockchain locally for fast testing.
- Provides instant transaction confirmations and customizable network settings.
- Supports debugging and fork testing.

INHERITANCE IN SOLIDITY

Solidity supports inheritance similar to C or Python language. Inheritance in Solidity allows a contract to inherit properties and functions from another contract, enabling code reuse and modular design.

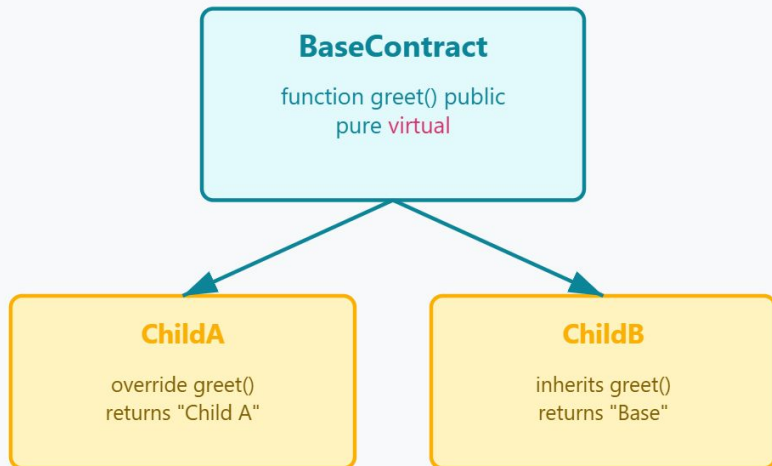
The contract that inherits from another is called the derived contract while the latter is called the base contract. A derived contract inherits all the public and internal functions and state variables from a base contract

Contract A can inherit from contract B using the **is** keyword. The function that is going to be overridden by a derived contract must be declared as **virtual** and the function that is going to override a Base function must use the keyword **override**.

Solidity supports multiple inheritance including Polymorphism and it is possible to call functions further up in the inheritance hierarchy internally by explicitly specifying the contract using `ContractName.functionName()` or using `super.functionName()` if you want to call the function one level higher up.

Inheritance Example

Solidity Inheritance (Practical Example)



- `virtual` allows functions to be overridden.
- `ChildA` overrides `greet()` with its own version.
- `ChildB` uses `greet()` directly from `BaseContract`.

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract BaseContract {
    function greet() public pure virtual returns (string memory) {
        return "Hello from Base";
    }
}

contract ChildA is BaseContract {
    function greet() public pure override returns (string memory) {
        return "Hello from Child A";
    }
}

contract ChildB is BaseContract {
    // No override, inherits BaseContract.greet()
}
  
```

SOLIDITY LIBRARIES

Solidity libraries are reusable, pre-written smart contracts designed to provide common functions that other contracts can use without duplicating code. It is designed to help you organize your code by putting commonly used functions in one place.

Libraries are:

- **Stateless:** Libraries cannot store data or hold Ether.
- **Reusable:** The same logic can be used in multiple contracts.
- **Efficient:** Functions can be internal or public. Functions can be inlined or deployed once and referenced.

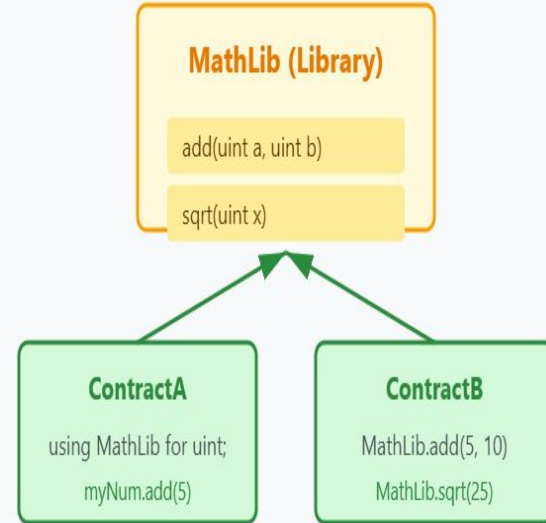
Overall Solidity libraries are useful for maths, conversions and string manipulations


```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 library MathLib {
5     function add(uint a, uint b) internal pure returns (uint) {
6         return a + b;
7     }
8
9     function sqrt(uint x) internal pure returns (uint) {
10         if (x == 0) return 0;
11         uint z = (x + 1) / 2;
12         uint y = x;
13         while (z < y) {
14             y = z;
15             z = (x / z + z) / 2;
16         }
17         return y;
18     }
19 }
20
21 contract ContractA {
22     using MathLib for uint;
23
24     function add(uint myNum) external pure returns (uint) {
25         return myNum.add(5);
26     }
27 }
28
29 contract ContractB {
30     function sqrt(uint a) external pure returns (uint) {
31         return MathLib.sqrt(a);
32     }
33
34     function add(uint a, uint b) external pure returns (uint) {
35         return MathLib.add(a,b);
36     }
37 }

```

Solidity Libraries



- Libraries contain reusable functions (stateless) that can be used by multiple contracts
- Contracts attach libraries using "using A for B" or call functions directly

SOLIDITY EVENTS

Solidity events are a way for smart contracts to communicate with the outside world by logging specific occurrences or state changes on the blockchain. They allow decentralized applications to track contract activity efficiently and provide a mechanism for external applications (e.g. frontends, indexers) to listen for updates.

They are defined using the **event** keyword and are triggered using the **emit** keyword.

Solidity Events

PaymentContract

```
function pay() public payable {
    emit PaymentReceived(msg.sender, msg.value);
}

event PaymentReceived(
    address indexed sender,
    uint amount
);
```

Transaction Receipt Logs

```
[0] PaymentReceived(
  sender: 0x5B3...d4a1 (indexed)
  amount: 1.5 ETH
)
```

- Events are blockchain logs that smart contracts can emit (costs gas)

SOLIDITY ERROR and ERROR HANDLING

Errors in Solidity provide a convenient and gas-efficient way to explain to the user why an operation failed.

They are mechanisms to handle exceptional conditions in smart contracts allowing contracts to communicate issues, revert transactions gracefully and save gas.

Errors can be defined inside and outside of contracts (including interfaces and libraries).

Types of Errors in Solidity

Type	Use Case	Refunds Gas?	Can Provide Message?
<code>require</code>	Input validation, preconditions	✓ Yes	✓ Yes
<code>revert</code>	Custom logic, nested conditions	✓ Yes	✓ Yes
<code>assert</code>	Internal consistency checks	✗ No	✗ No (message optional since 0.8)
<code>custom errors</code>	Gas-efficient, developer-defined errors	✓ Yes	✓ Via error name and data

The End

Questions?

ASSIGNMENT

Build a simple Vault system that allows users to deposit and withdraw Ether.

Requirements:

- Use a Library for math
- Emit an Event when a user deposits or withdraws.
- Use `require()` or `revert()` to prevent over-withdrawing or depositing 0 ETH
- Implement Inheritance:
 - Base contract: VaultBase (defines structure and shared logic)
 - Derived contract: VaultManager (implements deposit/withdraw functions)